



**UNIVERSITI SAINS MALAYSIA**

School of Electrical and Electronics Engineering  
Universiti Sains Malaysia, Engineering Campus

SEMESTER 1, 2026/2027

## **EEM 441**

Control & Instrumentation Laboratory

### **Experiment 4**

LCD INTERFACING TO ALTERA DE2 BOARD, AND STEPPER  
MOTOR CONTROL WITH FPGA

Date : \_\_\_\_\_

Group No. : \_\_\_\_\_

Group Members : \_\_\_\_\_ Matrix # \_\_\_\_\_

: \_\_\_\_\_ Matrix # \_\_\_\_\_

: \_\_\_\_\_ Matrix # \_\_\_\_\_

: \_\_\_\_\_ Matrix # \_\_\_\_\_

## Experiment 4

# LCD Interfacing to Altera DE2 Board, and Stepper Motor Control with FPGA

### Objectives

---

1. To use Verilog code as a design entry method in Altera Quartus II Software.
2. To interface an LCD module to the Altera DE2 board.
3. To design digital circuits in an FPGA to control a stepper motor.
4. To simulate the circuit using Altera Quartus II Software.
5. To download the design onto an Altera DE2 board.

### Equipment / Software Required

---

1. Altera Quartus II software
  - version 9/10/12 for Cyclone II
  - version 9/10/12/latest for Cyclone IV
2. Altera DE2 board
3. Parallax 4-phase, 12 V, 75  $\Omega$  unipolar stepper motor
4. ULN2003 Darlington transistor array (motor driver)

### Introduction

---

In recent years, LCDs have largely replaced simple LED displays: they can show numbers, characters, and graphics, and are relatively easy to program. Table 4.1 lists the LCD pins relevant to programming.

The *enable* (E) pin is used by the LCD to latch the information presented on its data pins: a high-to-low transition on E must be applied for the LCD to latch the data, and this pulse must be at least 450 ns wide. The required timing is shown in Figure 4.1.

Two important internal registers are selected via the RS pin. When RS = 0, the *instruction/command* register is selected, allowing commands to be sent to the LCD (Table 4.2 lists the available command codes). When RS = 1, the *data* register is selected, allowing ASCII character data to be written for display, sent through data pins D0–D7.

It is good practice to check the busy flag before writing to the LCD: with RS = 0 and  $\overline{R/\overline{W}} = 1$ , the busy flag appears on D7. While D7 = 1 the LCD is busy with internal operations and

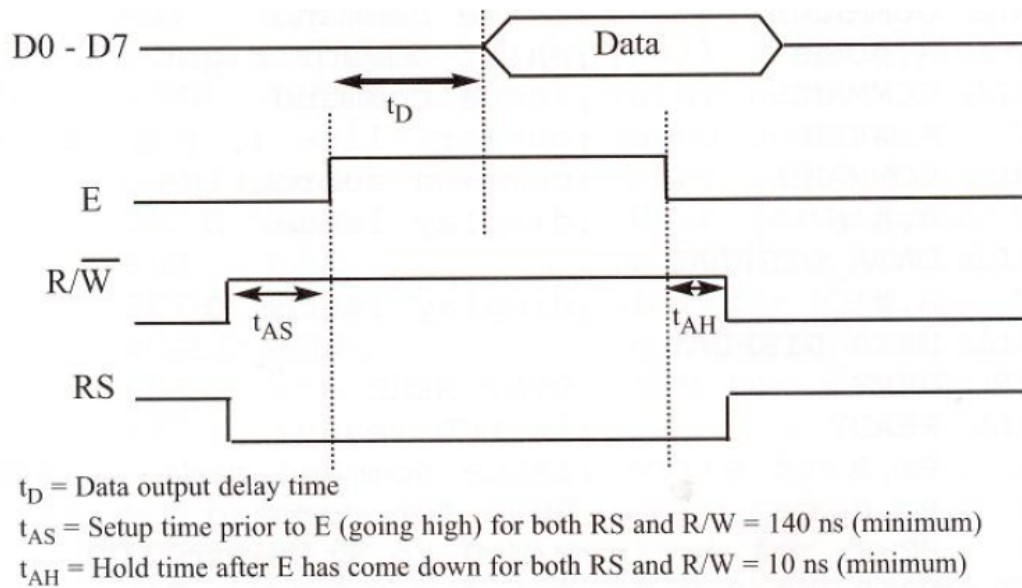
will not accept new data; once  $D7 = 0$ , the LCD is ready to receive the next byte.

Unlike an ordinary DC motor, a stepper motor does not spin freely when power is applied — it must be driven with a continuously pulsed sequence of specific coil-energising patterns. Each pulse advances the rotor by a fixed step angle (for this motor,  $3.6^\circ$  per step), which is what gives stepper motors their positioning precision: as long as speed and torque stay within the motor's limits, the controller always knows the rotor's exact position.

The rotor is driven by the interaction of magnetic fields as coils are switched on and off in sequence. Table 4.3 shows the normal four-coil unipolar step sequence; taking a step means driving two of the four coil-driver pins high at a time, in the order shown, through the ULN2003 driver.

Table 4.1: Pin description for the LCD module.

| Pin | Symbol           | I/O | Description                                     |
|-----|------------------|-----|-------------------------------------------------|
| 1   | $V_{ss}$         | —   | Ground                                          |
| 2   | $V_{cc}$         | —   | +5 V power supply                               |
| 3   | $V_{ee}$         | —   | Power supply to control contrast                |
| 4   | RS               | I   | RS = 0: command register; RS = 1: data register |
| 5   | $R/\overline{W}$ | I   | 0: write; 1: read                               |
| 6   | E                | I/O | Enable (latches data on high-to-low transition) |
| 7   | DB0              | I/O | 8-bit data bus                                  |
| 8   | DB1              | I/O |                                                 |
| 9   | DB2              | I/O |                                                 |
| 10  | DB3              | I/O |                                                 |
| 11  | DB4              | I/O |                                                 |
| 12  | DB5              | I/O |                                                 |
| 13  | DB6              | I/O |                                                 |
| 14  | DB7              | I/O |                                                 |



Note: Read requires an L-to-H pulse for the E pin.

Figure 4.1: LCD write-cycle timing diagram.



Table 4.2: LCD command codes.

| Code (hex) | Command to LCD instruction register   |
|------------|---------------------------------------|
| 1          | Clear display screen                  |
| 2          | Return home                           |
| 4          | Decrement cursor (shift cursor left)  |
| 6          | Increment cursor (shift cursor right) |
| 5          | Shift display right                   |
| 7          | Shift display left                    |
| 8          | Display off, cursor off               |
| A          | Display off, cursor on                |
| C          | Display on, cursor off                |
| E          | Display on, cursor blinking           |
| F          | Display on, cursor blinking           |
| 10         | Shift cursor position left            |
| 14         | Shift cursor position right           |
| 18         | Shift entire display left             |
| 1C         | Shift entire display right            |
| 80         | Force cursor to beginning of line 1   |
| C0         | Force cursor to beginning of line 2   |
| 38         | 2 lines, 5 × 7 dot matrix             |

## Introduction

Unlike an ordinary DC motor, a stepper motor does not spin freely when power is applied — it must be driven with a continuously pulsed sequence of specific coil-energising patterns. Each pulse advances the rotor by a fixed step angle (for this motor, 3.6° per step), which is what gives stepper motors their positioning precision: as long as speed and torque stay within the motor's limits, the controller always knows the rotor's exact position.

The rotor is driven by the interaction of magnetic fields as coils are switched on and off in sequence. Table 4.3 shows the normal four-coil unipolar step sequence; taking a step means driving two of the four coil-driver pins high at a time, in the order shown, through the ULN2003 driver.

Table 4.3: Normal step sequence for a four-coil unipolar stepper motor.

| Coil            | Step 1 | Step 2 | Step 3 | Step 4 | Step 1 |
|-----------------|--------|--------|--------|--------|--------|
| 1 (B)           | 1      | 1      | 0      | 0      | 1      |
| 2 ( $\bar{B}$ ) | 0      | 0      | 1      | 1      | 0      |
| 3 (A)           | 1      | 0      | 0      | 1      | 1      |
| 4 ( $\bar{A}$ ) | 0      | 1      | 1      | 0      | 0      |

## Procedure

### Activity 1 – Displaying lookup-table data on the LCD

*Objective: display fixed text, stored in a lookup table, on the LCD.*

1. Open Quartus II (Device family: Cyclone II (EP2C35F672C6N) or Cyclone IV (EP4CE115F29C7N)).
2. Write the Verilog modules given in Listing 4.1 (lcd\_t) and Listing 4.2 (LCD\_Controller) to send commands and data to the LCD.
3. lcd\_t uses a lookup table (LUT) to store the sequence of commands and characters to send to the LCD.
4. Create a reusable symbol for each module, as in Experiment 3.
5. Draw the schematic in Figure 4.2, connecting lcd\_t's outputs to LCD\_Controller's inputs, and LCD\_Controller's outputs to the LCD interface pins.
6. Assign pin numbers as given in the Altera DE2 board manual (Cyclone II or Cyclone IV).
7. Download the program to the Altera DE2 board.
8. Discuss the result based on your observation of the LCD.

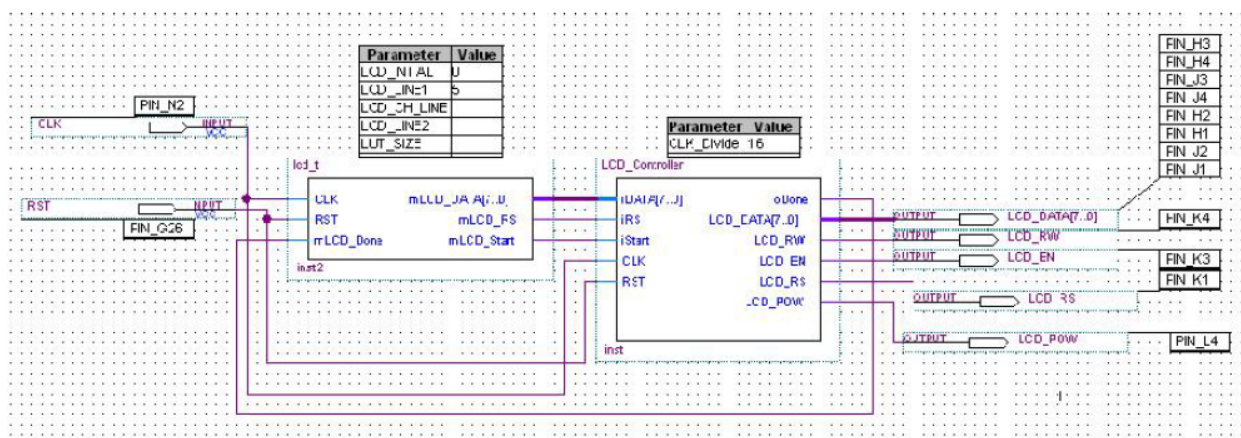


Figure 4.2: Schematic diagram for the LCD interface: lcd\_t (lookup-table sequencer) feeding LCD\_Controller (bus-timing controller).

Table 4.4: Pin assignment for the LCD interface.

| #  | Node name   | Direction | Location |
|----|-------------|-----------|----------|
| 1  | LCD_DATA[7] | Output    |          |
| 2  | LCD_DATA[6] | Output    |          |
| 3  | LCD_DATA[5] | Output    |          |
| 4  | LCD_DATA[4] | Output    |          |
| 5  | LCD_DATA[3] | Output    |          |
| 6  | LCD_DATA[2] | Output    |          |
| 7  | LCD_DATA[1] | Output    |          |
| 8  | LCD_DATA[0] | Output    |          |
| 9  | LCD_EN      | Output    |          |
| 10 | LCD_POW     | Output    |          |
| 11 | LCD_RS      | Output    |          |
| 12 | LCD_RW      | Output    |          |
| 13 | CLK         | Input     |          |
| 14 | RST         | Input     |          |

## lcd\_t.v — Appendix 1: lookup-table sequencer (part 1 of 2)

```

1  module lcd_t (
2      // Host side
3      CLK, RST, mLCD_Done,
4      mLCD_DATA, mLCD_RS, mLCD_Start
5  );
6      // Host side
7      input  CLK, RST, mLCD_Done;
8      // Internal wires/registers
9      output [7:0] mLCD_DATA;
10     output mLCD_RS, mLCD_Start;
11     reg  [5:0] LUT_INDEX;
12     reg  [8:0] LUT_DATA;
13     reg  [5:0] mLCD_ST;
14     reg  [17:0] mDLY;
15     reg  mLCD_Start;
16     reg  [7:0] mLCD_DATA;
17     reg  mLCD_RS;
18     wire mLCD_Done;
19
20     parameter LCD_INITIAL = 0;
21     parameter LCD_LINE1   = 5;
22     parameter LCD_CH_LINE = LCD_LINE1 + 16;
23     parameter LCD_LINE2   = LCD_LINE1 + 16 + 1;

```

```

24     parameter LUT_SIZE      = LCD_LINE1 + 32 + 1;
25
26     // Sequencing state machine: send one LUT entry, wait for
27     // LCD_Controller to finish (mLCD_Done), pause, then advance.
28     always @(posedge CLK or negedge RST)
29     begin
30         if (!RST)
31         begin
32             LUT_INDEX  <= 0;
33             mLCD_ST     <= 0;
34             mDLY        <= 0;
35             mLCD_Start  <= 0;
36             mLCD_DATA   <= 0;
37             mLCD_RS     <= 0;
38         end
39         else if (LUT_INDEX < LUT_SIZE)
40         begin
41             case (mLCD_ST)
42             0: begin
43                 mLCD_DATA <= LUT_DATA[7:0];
44                 mLCD_RS   <= LUT_DATA[8];
45                 mLCD_Start <= 1;
46                 mLCD_ST   <= 1;
47             end
48             1: if (mLCD_Done) begin
49                 mLCD_Start <= 0;
50                 mLCD_ST   <= 2;
51             end
52         end

```

#### lcd\_t.v — Appendix 1: lookup-table sequencer (part 2 of 2, continued)

```

1         2: if (mDLY < 18'h3FFFE)
2             mDLY <= mDLY + 1;
3         else begin
4             mDLY <= 0;
5             mLCD_ST <= 3;
6         end
7         3: begin
8             LUT_INDEX <= LUT_INDEX + 1;
9             mLCD_ST   <= 0;
10        end
11    endcase
12    end
13    end
14    // Lookup table: LCD_INITIAL entries are setup commands
15    // (function set, display on, clear, entry mode, home);
16    // LCD_LINE1 / LCD_LINE2 entries are ASCII characters for
17    // each line, with the MSB (bit 8) marking RS = 1 (data).
18    always
19    begin
20        case (LUT_INDEX)

```

```
21 LCD_INITIAL+0: LUT_DATA <= 9'h038; // function set
22 LCD_INITIAL+1: LUT_DATA <= 9'h00C; // display on
23 LCD_INITIAL+2: LUT_DATA <= 9'h001; // clear display
24 LCD_INITIAL+3: LUT_DATA <= 9'h006; // entry mode set
25 LCD_INITIAL+4: LUT_DATA <= 9'h080; // cursor home, line 1
26 // Line 1: "Welcome to the "
27 LCD_LINE1+0: LUT_DATA <= 9'h120;
28 LCD_LINE1+1: LUT_DATA <= 9'h157;
29 LCD_LINE1+2: LUT_DATA <= 9'h165;
30 // ... remaining characters follow the same pattern ...
31 LCD_LINE1+15: LUT_DATA <= 9'h120;
32 LCD_CH_LINE: LUT_DATA <= 9'h0C0; // move to line 2
33 // Line 2: " EEM 441 Lab "
34 LCD_LINE2+0: LUT_DATA <= 9'h120;
35 LCD_LINE2+4: LUT_DATA <= 9'h145;
36 // ... remaining characters follow the same pattern ...
37 LCD_LINE2+15: LUT_DATA <= 9'h120;
38 default: LUT_DATA <= 9'h000;
39 endcase
40 end
41 endmodule
42
```

Listing 4.1: lcd\_t.v – lookup-table LCD message sequencer.

## LCD\_Controller.v – Appendix 2: bus-timing controller (part 1 of 2)

```

1  module LCD_Controller (
2      // Host side
3      iDATA, iRS, iStart, oDone, CLK, RST,
4      // LCD interface
5      LCD_DATA, LCD_RW, LCD_EN, LCD_RS, LCD_POW
6  );
7      parameter CLK_Divide = 16;
8
9      // Host side
10     input  [7:0] iDATA;
11     input  iRS, iStart;
12     input  CLK, RST;
13     output reg oDone;
14
15     // LCD interface
16     output [7:0] LCD_DATA;
17     output reg LCD_EN;
18     output LCD_RW;
19     output LCD_RS;
20     output LCD_POW;
21
22     // Internal registers
23     reg [4:0] Cont;
24     reg [1:0] ST;
25     reg preStart, mStart;
26
27     // Write-only interface: bypass iRS straight to LCD_RS
28     assign LCD_DATA = iDATA;
29     assign LCD_RW    = 1'b0;
30     assign LCD_RS    = iRS;
31     assign LCD_POW   = 1'b1;
32     always @(posedge CLK or negedge RST)
33     begin
34         if (!RST)
35         begin
36             oDone    <= 1'b0;
37             LCD_EN    <= 1'b0;
38             preStart <= 1'b0;
39             mStart    <= 1'b0;
40             Cont      <= 0;
41             ST        <= 0;
42         end
43         else
44         begin
45             // Rising-edge detect on iStart
46             preStart <= iStart;
47             if ({preStart, iStart} == 2'b01)
48             begin
49                 mStart <= 1'b1;
50                 oDone  <= 1'b0;

```

```
51     end
```

#### LCD\_Controller.v – Appendix 2: bus-timing controller (part 2 of 2, continued)

```

1      if (mStart)
2      begin
3          case (ST)
4              0: ST <= 1;                // wait setup
5              1: begin
6                  LCD_EN <= 1'b1;        // raise enable
7                  ST      <= 2;
8              end
9              2: if (Cont < CLK_Divide)    // hold enable pulse width
10                 Cont <= Cont + 1;
11                 else
12                     ST <= 3;
13             3: begin
14                 LCD_EN <= 1'b0;          // drop enable: latches data
15                 mStart <= 1'b0;
16                 oDone  <= 1'b1;
17                 Cont   <= 0;
18                 ST     <= 0;
19             end
20         endcase
21     end
22 end
23 end
24 endmodule
```

Listing 4.2: LCD\_Controller.v – generates the LCD enable-pulse timing.

### Activity 2 – Displaying DIP switch input on the LCD

*Objective: display the live 8-bit DIP switch configuration on the LCD.*

1. Write Verilog code that reads the 8-bit DIP switch configuration and displays it on the LCD (e.g. if switch 1 is turned on, the LCD should display 1 in the corresponding position).
2. Print and submit the Verilog code and simulation result.

### Activity 3 – Verilog stepper motor controller

*Objective: write Verilog code to control the motion of a stepper motor.*

1. Open Quartus II and start the New Project Wizard.
  - Working directory: fpga; project name: activity1; top-level entity: step.
  - Device family: Cyclone II (EP2C35F672C6N) or Cyclone IV (EP4CE115F29C7N).
  - Accept the default EDA tool settings and finish.

2. Create a new Verilog HDL file and enter the code in Listing 4.3 (step, divider, and stepper modules).
3. Save as step and start compilation.
4. Use **Tools** → **Netlist Viewer** → **RTL Viewer** to generate the block overview shown in Figure 4.3.
5. Assign pin numbers as shown in Table 4.5 (refer to the DE2 user manual). For the phaseout outputs that interface with the stepper motor driver, use expansion header GPIO\_0.
6. Download the program to the Altera DE2 board.
7. Discuss the result.

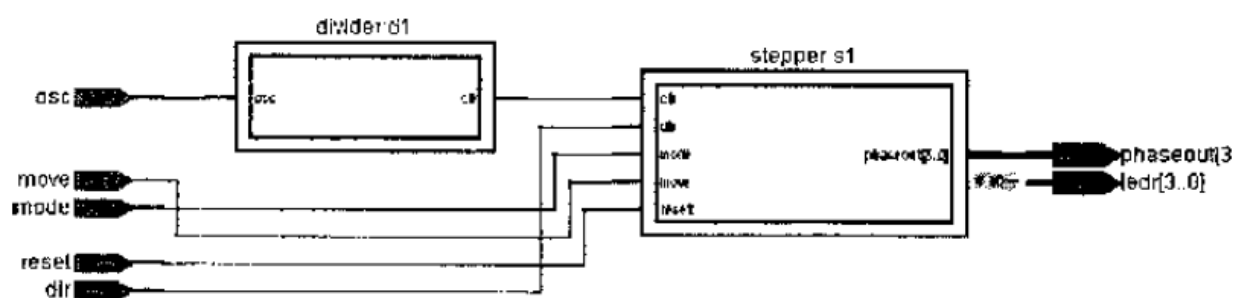


Figure 4.3: RTL Viewer overview of the step top-level module.

Table 4.5: Pin assignment for stepper motor control.

| # | Node name   | Direction | Pin location | # | Node name   | Direction | Pin location |
|---|-------------|-----------|--------------|---|-------------|-----------|--------------|
| 1 | osc         | Input     |              | 5 | ledr[0]     | Output    |              |
|   | dir (SW0)   | Input     |              | 6 | phaseout[3] | Output    |              |
|   | mode (SW1)  | Input     |              | 7 | phaseout[2] | Output    |              |
|   | move (SW2)  | Input     |              | 8 | phaseout[1] | Output    |              |
|   | reset (SW3) | Input     |              | 9 | phaseout[0] | Output    |              |
| 2 | ledr[3]     | Output    |              |   |             |           |              |
| 3 | ledr[2]     | Output    |              |   |             |           |              |
| 4 | ledr[1]     | Output    |              |   |             |           |              |

## step.v – top-level module

```

1  //=====
2  // Stepper Motor Controller
3  //=====
4  // TOP LEVEL MODULE
5  // reset (active low) brings the shaft to its reference position.
6  // mode selects step (1) or continuous (0) operation.
7  // dir selects clockwise (1) or anti-clockwise (0) motion.
8  // move shifts the shaft by one half-step in step mode.
9  module step(osc, reset, dir, mode, move, phaseout, ledr);
10     input  osc, reset, mode, dir, move;
11     output [3:0] phaseout, ledr;
12
13     wire clk;
14
15     divider d1 (
16         .osc (osc),
17         .clk (clk)
18     );
19
20     stepper s1 (
21         .clk      (clk),
22         .reset     (reset),
23         .dir       (dir),
24         .mode      (mode),
25         .move      (move),
26         .phaseout  (phaseout)
27     );
28 endmodule

```

## step.v – motor controller module (part 1 of 2)

```

1  // Module stepper is the motor controller.
2  module stepper(clk, reset, dir, mode, move, phaseout);
3      input  clk, reset, dir, mode, move;
4      output [3:0] phaseout;
5      reg [3:0] phaseout, position, ledr;
6
7      always @(posedge clk or negedge reset)
8      begin
9          if (reset == 0) begin
10             // motor resets to phase 1000 when reset is asserted
11             phaseout = 4'b1000;
12             ledr      = phaseout;
13         end
14         else begin
15             if (mode == 0) begin
16                 // Mode = 0: continuous motion.
17                 // The 8-state sequence below follows Table 1 in both
18                 // directions; only the first two states are shown here,
19                 // the remaining six states (0100, 0110, 0010, 0011,
20                 // 0001, 1001) follow the identical if(dir==0)/else
21                 // pattern, cycling back to 1000.
22                 case (phaseout)
23                     4'b1000: if (dir == 0) begin
24                         phaseout = 4'b1100; ledr = phaseout;
25                     end else begin
26                         phaseout = 4'b1001; ledr = phaseout;
27                     end
28                     4'b1100: if (dir == 0) begin
29                         phaseout = 4'b0100; ledr = phaseout;
30                     end else begin
31                         phaseout = 4'b1000; ledr = phaseout;
32                     end
33                     // ... 4'b0100, 4'b0110, 4'b0010, 4'b0011,
34                     //      4'b0001, 4'b1001 follow the same pattern ...
35                     default: $display("invalid input");
36                 endcase
37             end
38             else begin
39                 // Mode = 1: single-step motion, advances only when
40                 // move == 0. Uses the same 8-state sequence as above,
41                 // indexed by `position` instead of `phaseout`.
42                 if (move == 1'b0) begin
43                     case (position)
44                         // ... identical 8-state sequence as continuous
45                         //      mode, see Table 1 ...
46                         default: $display("invalid input");
47                     endcase
48                 end
49             end
50         end
51     end

```

```
51 end
```

#### step.v — motor controller module (part 2 of 2)

```
1  // Motor advances by one half-step on the falling edge of move.
2  always @(negedge move or negedge reset)
3  begin
4      if (reset == 0) begin
5          position = 4'b1000; // NOTE: corrected from the original
6                              // listing, which printed this value
7                              // as "4.101011" -- likely a
8                              // transcription error; 4'b1000
9                              // matches the reset state used
10                             // consistently elsewhere.
11          ledr = position;
12      end
13      else if (reset == 1) begin
14          position = phaseout;
15          ledr      = phaseout;
16      end
17  end
18 endmodule
```

#### step.v — clock divider module

```
1  // Module divider divides the incoming clock to produce a
2  // clock suitable for driving the motor controller.
3  module divider(osc, clk);
4      input  osc;
5      output clk;
6      reg    clk;
7      reg [15:0] count;
8
9      initial count = 16'b0000000000000000;
10
11     always @(posedge osc)
12     begin
13         count = count + 1;
14         clk    = count[15];
15     end
16 endmodule
```

Listing 4.3: step.v — top-level, motor-controller, and clock-divider modules.

### Activity 4 — Variable-speed control with display feedback

*Objective: extend the controller with selectable speed and direction/mode feedback.*

1. Write Verilog code that allows selection of three different speeds for the stepper motor in continuous-motion mode.
2. Display the motor's direction of rotation on the seven-segment display, and the current mode (step / continuous) on the LCD display.

3. Print and submit the Verilog code and schematic diagram.

## Appendix: Stepper Motor Datasheet Reference

Relevant specifications and wiring information for the Parallax #27964 stepper motor used in this experiment, extracted from the manufacturer datasheet (see References).



Figure 4.4: Parallax #27964 stepper motor.

Table 4.6: Technical specifications, Parallax #27964 stepper motor.

| Parameter                        | Value                  |
|----------------------------------|------------------------|
| Type                             | 4-phase, 12 V unipolar |
| Step angle                       | 3.6° per step          |
| Phase resistance                 | 75 $\Omega$            |
| Current                          | 150 mA                 |
| Phase inductance                 | 39 mH                  |
| Detent torque                    | 80 g·cm                |
| Holding torque                   | 600 g·cm               |
| Mounting hole spacing (diagonal) | 1.73 in                |
| Mounting hole diameter           | 0.11 in                |
| Shaft diameter                   | 0.197 in               |
| Shaft length                     | 0.43 in                |
| Motor diameter                   | 1.66 in                |
| Motor height                     | 1.35 in                |
| Weight                           | 0.55 lb                |

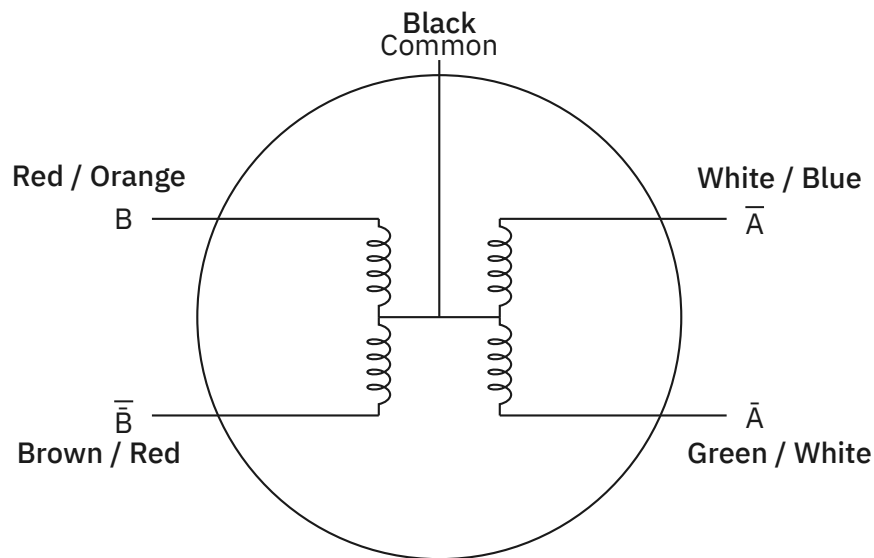


Figure 4.5: Coil winding and lead-wire colours, Parallax #27964 stepper motor.

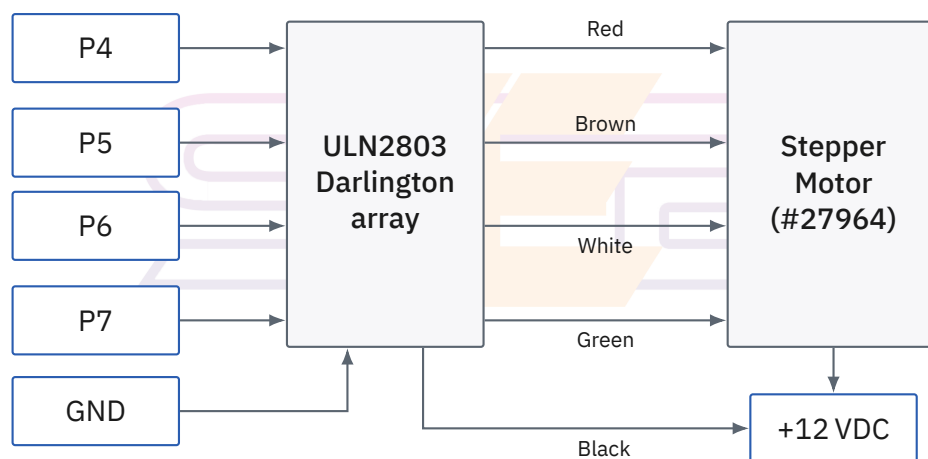


Figure 4.6: Reference driver-wiring diagram.

Note:

1. The P4–P7 labelling above follows the datasheet’s original microcontroller example; in this experiment, the corresponding driver inputs are instead the phaseout [3:0] outputs of the stepper module, assigned to DE2 GPIO pins as listed in Table 4.5.
2. The GND are to be common ground with the DE2 board, ULN2803 and the power source.
3. The +12 VDC is to be supplied from an external power source, not the DE2 board.

## References

1. Altera DE2 User Manual.
2. *Introduction to Quartus II Software – Verilog, VHDL, Schematic*, Altera University Program.

3. *Introduction to Verilog*, Carleton University course notes.
4. *Stepper Motor Controller Using MAX II CPLDs*, Altera Application Note AN-488.
5. Parallax Inc., *4-Phase 12-Volt 75  $\Omega$  Unipolar Stepper Motor (#27964)*, technical datasheet.

